Lesson plan · for the teacher

Leading computing with few devices: unit check

Programming · Level 4 · Unit check

Time	Materials
45 min	Board, chalk, paper. No devices needed.

Learning goals

- Unplugged, then plugged
- One phone, many pupils
- Code on paper
- A scheme of work

Activity: Team quiz

1. Make teams of four, with the four roles
2. Read each question aloud; teams agree on one answer
3. Reveal the answer and ask one team to explain why
4. Swap roles after every question

Discussion

- Which idea will you put into practice first?
- Which question surprised you, and why?
- Who in your school will you share this with?

Exit ticket

- One change I will lead in our school's computing this term

Try it in class tomorrow

Choose one idea from this course (an unplugged-then-plugged pair, a one-phone rota, a paper-first lesson or a one-page term plan) and put it into practice this month. Then share what happened with a colleague.

Worksheet · for pupils

Leading computing with few devices: unit check

Programming · Level 4 · Unit check

Name _____ Class _____ Date _____

1 **What is the best reason to start a computing topic unplugged?**

- ◯ It saves the phones' batteries
- ◯ Pupils meet the idea itself, without syntax in the way
- ◯ It replaces coding altogether

2 **In pair programming, the driver and the navigator should swap roles every 5 to 10 minutes.**

- ◯ True
- ◯ False

3 **Paper strips: the program counts down from 5 to 1, then prints Lift off!. One strip doesn't belong.**

Number the lines in order (1, 2, 3…). Cross out any line that doesn't belong.

- ☐ `n = n + 1`
- ☐ `print(n)`
- ☐ `n = 5`
- ☐ `while n > 0:`
- ☐ `n = n - 1`
- ☐ `print("Lift off!")`

4 **During live coding, pupils should spot this mistake. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  light = input("Light? ")
2  if light = "red":
3      print("Stop")
```

5 **Match each paper exercise to what it shows you about a pupil.**

Parsons strips	• • • •	Whether they can read a whole program
A trace table	• • • •	Whether they read code closely, and which misconceptions they hold
Find the bug	• • • •	How they think variables change
Predict the output	• • • •	Whether they understand order and indentation